

This is the final lecture on Convolution. In this lecture we will focus on dimension change in CNNs and number of parameters per model. We go one step at a time dealing one layer at a time. We will not go into super complicated zones and just stick to the basic hyperparameters.

Conv layers —

Hyperparameters to be covered —

- ① C_{in} , $n(\text{in-channels})$ or number of kernels per filter $\leftarrow$ affects param count
- ② C_{out} , $n(\text{out-channels})$ or number of filters $\leftarrow$ affects param count & output dims
- ③ k , kernel size (we will assume square kernel for 2D variant) $\leftarrow$ affects param count & output dims
- ④ s , stride value $\leftarrow$ affects output dims
- ⑤ padding, amount of 0 padding (can be a number or $\text{str} \in \{\text{'valid'}, \text{'same'}\}$) $\leftarrow$ affects output dims
- ⑥ bias, can be True or False $\leftarrow$ affects param count

Now let's build a formula for param count of a conv. layer. Assuming bias = False —

$$\text{param count} = n(\text{points per kernel}) \times n(\text{kernels per filter}) \times n(\text{filters})$$

$$= \begin{cases} k \times C_{in} \times C_{out} & \text{for 1D} \\ k^2 \times C_{in} \times C_{out} & \text{for 2D since we are using square kernels} \end{cases}$$

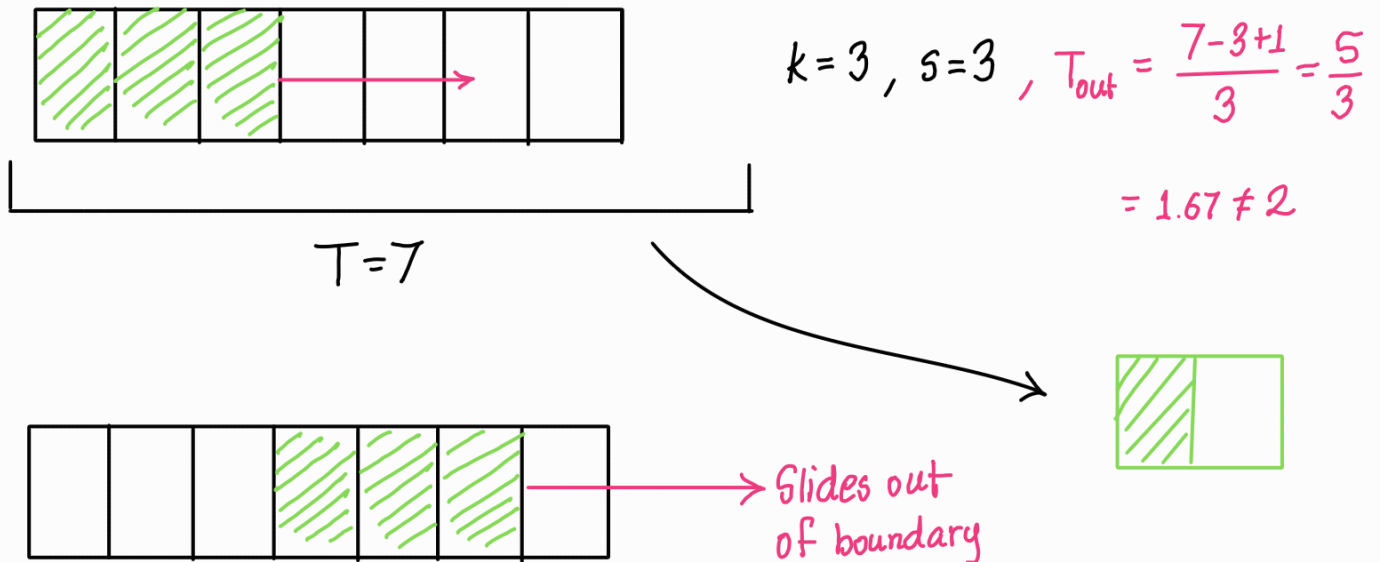
Making bias = True adds one bias parameter per filter, so the param counts become —

$$\text{param count} = \begin{cases} k \times C_{in} \times C_{out} + \text{bias} \times C_{out} & \text{for 1D} \\ k^2 \times C_{in} \times C_{out} + \text{bias} \times C_{out} & \text{for 2D} \end{cases}$$

Now let's move on to the dimension change. We know a regular conv block uses a truncated convolution which results in the dimension to decrease by $(k-1)$.

But we didn't bother with stride before. Stride is equivalent to skip rate of signal data. So output shape of the signal is basically given by $(T-k+1)/s$.

But this formula can give you a float value. Assume the following example —



So we somehow have to update our formula.

Our current formula underestimates number of patches at times because the numerator is the output shape while the patches are being made from the input shape. Basically a ceiling function should resolve the issue —

$$T_{out} = \left\lceil \frac{T_{in} - k + 1}{s} \right\rceil$$

In literature a different variant of the formula is used —

$$T_{out} = \left\lfloor \frac{T_{in} - k}{s} + 1 \right\rfloor$$

The 2 formulas are equivalent.

Proof: $\left\lceil \frac{n}{m} \right\rceil = \left\lfloor \frac{n-1}{m} \right\rfloor + 1$ if m is positive. Since $s \geq 1$, $\left\lceil \frac{T_{in} - k + 1}{s} \right\rceil = \left\lfloor \frac{T_{in} - k}{s} \right\rfloor + 1$

Now, for any x and $n : x \in \mathbb{R}$ and $n \in \mathbb{Z}$, $\lfloor x+n \rfloor = \lfloor x \rfloor$

$$\therefore \left\lfloor \frac{T_{in}-k}{s} \right\rfloor + 1 = \left\lfloor \frac{T_{in}-k}{s} + 1 \right\rfloor$$

$$\text{So, } T_{out} = \left\lfloor \frac{T_{in}-k+1}{s} \right\rfloor = \left\lfloor \frac{T_{in}-k}{s} + 1 \right\rfloor$$

Now let's include padding to the mix. Padding is used to minimize dimension loss by going a bit more towards regular convolution.

Regular signal —



Padding = 1 —

→ 1 point padding on both sides



Padding doesn't have to do with 0s. There are other methods of padding. But 0 padding is the most common.

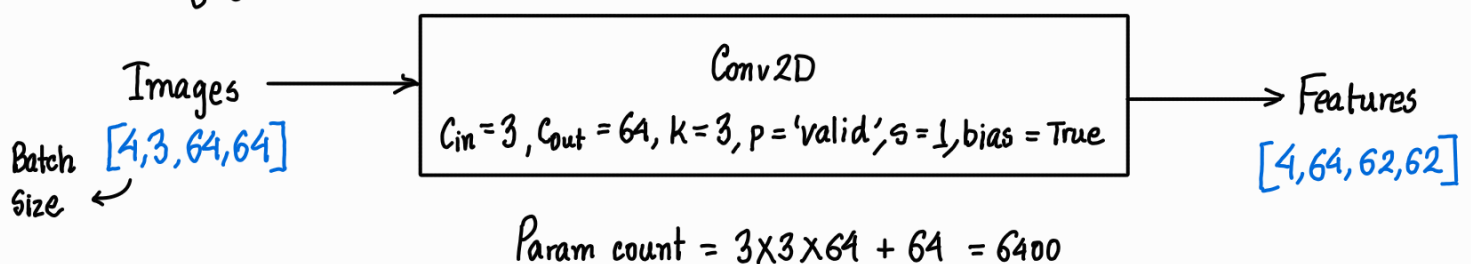
If padding = 'valid', no padding is added. If padding = 'same', $L_{in} = L_{out}$. The padding length is automatically set by the framework.

When padding is added to the model,

$$T_{out} = \left\lfloor \frac{T_{in}+2P-k+1}{s} \right\rfloor = \left\lfloor \frac{T_{in}+2P-k}{s} + 1 \right\rfloor$$

There is also a dilation, d , hyperparameter but we will not study that.

So let's say you have 4 images that you are passing through a Conv2D block —

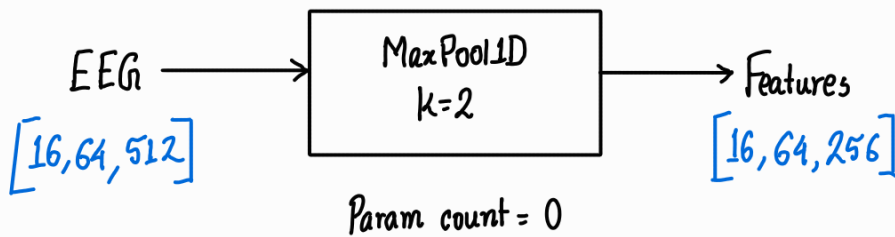


Pooling layers —

Pooling layers are special conv layers generally used to downscale the data. The dim change formula is the same as regular conv. layers. However, pooling is done per channel and there are no C_{in} or C_{out} hyper parameters. By default stride == k unless otherwise specified. By default padding is kept 0. Pooling doesn't support str. There is also no bias term as pooling layers are not learnable.

Pooling layers do not have any trainable parameters.

Assume we have 16 EEG signals (64 channels) that we are passing through a MaxPool1D



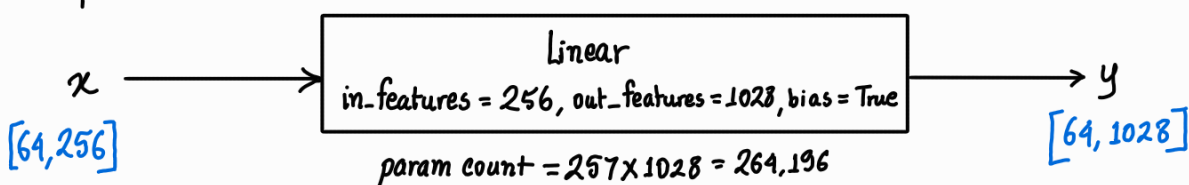
Linear or FC layers —

Linear layers have only 3 hyperparameters —

- ① in-features
- ② out-features
- ③ bias

$$\begin{aligned}\text{param count} &= n(\text{points in the transformation matrix}) + \text{bias} \cdot \text{out-features} \\ &= \text{in-features} \cdot \text{out-features} + \text{bias} \cdot \text{out-features}\end{aligned}$$

Example —

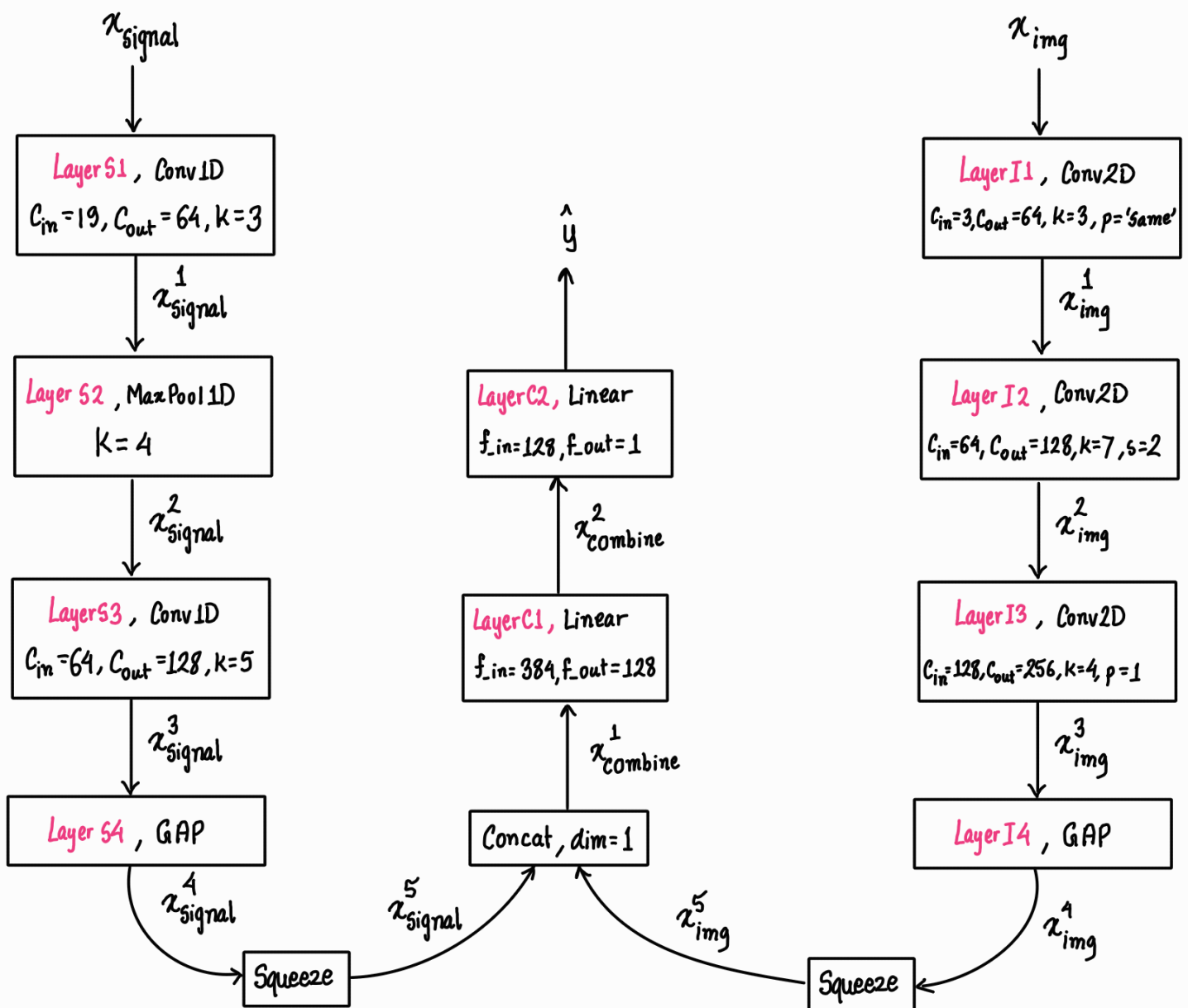


Complex Example —

Assume you are trying to map EEG readings (19 channels) to facial expressions. For this you have taken 5s of EEG readings from various subjects after artificially triggering the emotions. Midway through the readings, you take a snapshot of the subject's face and store it as an RGB image of dims 1024×1024 . The sampling rate of the EEG acquisition device is 256 Hz. You create a complex dataset having x_{signal} , x_{img} and y , where,

$$y = \begin{cases} 1 & \text{if } x_{\text{signal}} \text{ and } x_{\text{img}} \text{ belong to the same emotions} \\ 0 & \text{otherwise} \end{cases}$$

After you had created the dataset, you come up with a model that goes like —



Assume a batch size of 16, and fill up the param count and dim tables below. In case hyperparams are not provided, assume these defaults —

$\text{bias} = \text{True}$ for all layers

$p = \text{'valid'}$ for all conv and pool layers

$s = 1$ for conv layers and $s = k$ for pool layers

Param count table

Layer name	Param count
LayerS1	$19 \times 64 \times 3 + 64 = 3,712$
LayerS2	0
LayerS3	$64 \times 128 \times 5 + 128 = 41,088$
LayerS4	0
LayerI1	$3 \times 64 \times 3^2 + 64 = 1,792$
LayerI2	$64 \times 128 \times 7^2 + 128 = 401,536$
LayerI3	$128 \times 256 \times 4^2 + 256 = 524,544$
LayerI4	0
LayerC1	$384 \times 128 + 128 = 47,753$
LayerC2	$128 \times 1 + 1 = 129$
Total	1,020,554

Dimensions table

Variable	Shape
x_{signal}	$[16, 19, 1280]$; $T_{\text{in}} = 256 \times 5 = 1280$
x_{signal}^1	$[16, 64, 1278]$
x_{signal}^2	$[16, 64, 319]$; $T = \lfloor \frac{1278}{4} \rfloor$
x_{signal}^3	$[16, 128, 315]$
x_{signal}^4	$[16, 128, 1]$
x_{signal}^5	$[16, 128]$
x_{img}	$[16, 3, 1024, 1024]$
x_{img}^1	$[16, 64, 1024, 1024]$
x_{img}^2	$[16, 128, 509, 509]$
x_{img}^3	$[16, 256, 254, 254]$
x_{img}^4	$[16, 256, 1, 1]$
x_{img}^5	$[16, 256]$
x_{combine}^1	$[16, 384]$
x_{combine}^2	$[16, 128]$
$\hat{y}$	$[16, 1]$

In your exam there will be at least one question like this. I may not ask you for both tables. But at least one will be there.